

THE UNIVERSITY OF DAR ES SALAAM



**COLLEGE OF INFORMATION AND COMMUNICATION TECHNOLOGY
DEPARTMENT OF COMPUTER SCIENCE**

COURSE: IS 365 - PRACTICAL ASSIGNMENT (LOCAL LLM APPLICATION)

**UDSM STUDENT SUPPORT ASSISTANT
TECHNICAL REPORT**

LOCAL LLM PIPELINE - FASTAPI, STREAMLIT AND OLLAMA

STUDENT'S DETAILS:

NAME	REGISTRATION NUMBER	PROGRAMME
Charles J Tungaraza	2023-04-13490	BSc Computer Science
Stella Thomas Kahungo	2023-04-03737	BSc Computer Science
Godbless Kaaya	2023-04-03579	BSc Computer Science
Baraka Jimmy Maengesho	2023-04-06473	BSc Computer Science
Frank Elikana Wallace	2023-04-13642	BSc Computer Science

Kelvin George Msuya	2023-04-08929	BSc BIT
---------------------	---------------	---------

1. Introduction

This report describes our IS 365 group project: a **UDSM Student Support Assistant** - a chatbot that answers questions about campus services on one laptop using Python, FastAPI, Streamlit, and a local language model through Ollama (llama3.2:1b).

The assistant covers registration (ARIS), exams, library, ICT, hostel, fees (GePG), academic calendar, and student conduct. The goal was not the smartest possible chatbot, but a complete pipeline: setup → local model → API → web UI → tests and logs.

Objectives: (1) run an LLM locally, (2) expose it through a REST API, (3) build a simple chat UI, (4) improve answers with FAQ/prospectus RAG and a UDSM system prompt, (5) show tests, logging, and error handling.

Our Scope: one-machine prototype with FAQ content.

Out of our scope: real ARIS login, official policy hosting, HTTPS, GPUs, and model fine-tuning.

2. Problem and solution

Students often look for the same answers across websites, notice boards, and offices (registration, fees, hostel, exams). That wastes time, especially for new students.

Solution: a chat page where a student types a question and gets an answer on the same screen. The local model is guided by:

1. Curated FAQ (data/university_faq.md)
2. UDSM Undergraduate Prospectus extract (data/udsm_prospectus.md)
3. Keyword RAG before each answer
4. An improved UDSM-specific system prompt

Example question	Expected topic
How do I register for courses at UDSM?	ARIS registration
How do I pay fees through GePG?	Fee payment
What are UDSM library hours?	Library services
How do I apply for hostel?	Accommodation

The FAQ is **demo content** for the assignment, not official policy.

3. Tools and technologies

We developed on Windows 10 (Dell Inspiron 3543, 8 GB RAM). Choices favoured tools that fit low memory:

Tool	Role
Python 3.10+ / .venv	Language and isolation
Ollama + llama3.2:1b	Local LLM (~1.3 GB)
FastAPI + Uvicorn	REST API on port 8000
Streamlit	Chat UI on port 8501
httpx / requests	HTTP clients
pytest	Automated API tests
Markdown FAQ files	Knowledge base

We also added conversation history (last 3 turns), streaming answers, and Good/Average/Poor ratings. We did **not** use vector databases or embeddings (too heavy for 8 GB RAM).

4. System architecture

Three services run on one computer. The frontend never calls Ollama directly.

Part	Port	Role
Streamlit	8501	Student chat UI
FastAPI	8000	Validation, RAG, Ollama calls, logging
Ollama	11434	Text generation

Request flow: Student question → Streamlit POST /ask → FastAPI validates input → keyword RAG on FAQ/prospectus → prompt + context to Ollama → answer returned → logged to backend/logs/app.log → optional rating to data/feedback.jsonl.

Main files: backend/main.py, llm_client.py, rag.py, config.py, frontend/app.py, data/university_faq.md, data/udsm_prospectus.md.

API endpoints: GET /health, POST /ask, POST /ask/stream, POST /feedback, GET /feedback/summary, GET /prompts.

5. RAG and prompting

Keyword RAG lowercases the question, scores FAQ/prospectus sections by word overlap, and inserts the top chunks as context. FAQ sections cover registration, exams, library, ICT, hostel, fees, calendar, and conduct.

Original prompt: You are a helpful assistant.(too generic).

Improved prompt: names the UDSM assistant role, lists allowed topics, requires use of FAQ context, and forbids inventing fees or deadlines when context is missing.

Model settings: llama3.2:1b, num_ctx2048, num_predict320, timeout 120 s, max history 3 turns.

6. Implementation summary

Task	What we built
Environment	.venv, requirements.txt, start/stop batch scripts
Local LLM	Ollama + llama3.2:1b
Backend	FastAPI routes, Pydantic validation, logging
Frontend	Streamlit chat, RAG/history/stream toggles, ratings
Prompting	Original vs improved prompt in config.py
Errors	422 empty input, 503 Ollama down, connection errors in UI
Bonuses	RAG, ratings, history, streaming

Run (Windows): create venv → pip install -r requirements.txt → ollama pull llama3.2:1b → start Ollama, backend, and Streamlit (or use run_all.bat). Open <http://localhost:8501>.

7. Testing, errors, and challenges

Automated tests: pytest tests/ -v - 10 passed with Ollama running (health, validation, feedback, live /ask, history, streaming). Live Ollama calls made the suite slow (~80+ seconds on CPU).

Manual checks: Swagger /docs, /health, happy-path chat Q&A, backend-down error message, streaming, and log lines in app.log.

Error handling: empty question → 422; Ollama stopped → 503 / degraded health; backend stopped → clear Streamlit connection message; timeout → user-visible error after 120 s.

Challenges on 8 GB RAM: slow CPU answers (30–90 s) - mitigated with spinner/streaming and a timeout; port conflicts - mitigated with start/stop scripts; demo FAQ written by the group and labelled as non-official.

8. Production readiness and conclusion

This is a **local prototype**, not a campus production service. Before real use we would need HTTPS, student authentication, an official knowledge base, stronger servers, monitoring, rate limits, privacy rules for logs, and human escalation for uncertain answers.

Local vs cloud: local keeps data on the machine and avoids API cost, but is slower and uses a smaller model. Cloud APIs are faster and stronger but send student questions off-device.

Security today: no login, no HTTPS on localhost, plain-text logs, and possible hallucination even with RAG.

We successfully delivered a working self-hosted UDSM Student Support Assistant that meets the practical goals: local LLM, FastAPI backend, Streamlit UI, keyword RAG, improved prompting, pytest evidence, error handling, and logging.